

Support Ticket Triage Agent

Executive Report

Why we built this

Right now, someone has to read every support ticket by hand, work out what it's about, check whether a refund is owed, and write a reply. It's slow and it isn't always consistent from person to person. This agent takes over the first part — reading and sorting the ticket, and drafting a reply — but it never lets a refund of \$100 or more go through without a person signing off first.

How it works

A ticket comes in, gets checked for obvious problems (blank or broken input), gets sorted into a category, and then either gets handled automatically or gets held for a person to approve. That approval step is required for any big refund, and also for any ticket that looks like someone is trying to manipulate the system into approving something it shouldn't. Everything that happens gets logged, so there's a record to look back on.

Why we kept it simple

We didn't reach for a heavier framework like LangGraph or CrewAI here. Those earn their keep when an agent needs to loop back and retry itself, or when you've got multiple agents with different roles negotiating with each other. This isn't that — it's one path with one decision point. Plain Python code is easier to read, easier to test, and there's nothing hidden inside a framework that we'd need to explain later.

Did it actually work?

We ran it against 8 test tickets, including a blank one and one that tried to sneak in instructions to auto-approve a \$5,000 refund. Here's what came back:

What we checked	Result	In plain terms
Got the category right	8 out of 8	Every ticket was sorted correctly
Blocked manipulation attempts	8 out of 8	The fake refund request never got through
Speed	Basically instant	Well under a millisecond per ticket
Cost	\$0 for this step	Sorting is free; writing replies with AI will cost something
Handled bad input without crashing	8 out of 8	Blank tickets and a failed tool call were both handled cleanly
Left a clear trail	7 out of 8	Almost every result explains itself, so a person can double-check it

The one real problem we found: the way tickets get sorted is just keyword matching, with no sense of how confident it should be. We tested this on purpose with a garbled, nonsense ticket that happened to include the word “refund,” and it got sorted as a refund request just as confidently as a real one would be. The fix is straightforward — add a confidence check, and send anything unclear to a person instead of guessing.

What's not proven yet

- The sorting step has no confidence score, so a garbled ticket with the right keyword can fool it, the same way described above.
- The step where the agent actually writes a reply needs a live AI model and internet access. We didn't have that in this test environment, so it hasn't been tried for real yet — only everything up to that point has.
- The “human approval” step right now is just a function call standing in for the real thing. It's not yet hooked up to Slack, email, or whatever a reviewer would actually use.
- The tool that looks up a customer's priority level is a stand-in too. A real version would need proper retry logic if it fails, not just one attempt.

What we'd do next

- Add a confidence check to the sorting step, so unclear tickets go to a person instead of getting guessed at.
- Hook the approval step up to something real, like a Slack message a reviewer can approve or reject.
- Try the reply-writing step for real, with a live model and a proper monitoring setup. Start by just logging what it would send, without actually sending anything, until we trust it.
- Run a lot more than 8 test tickets — more variety, more languages, more weird edge cases — before letting this touch real customers.

Keeping it safe as it scales

As more tickets come through, the lookup and reply-writing calls should sit behind a proper queue with retries, instead of a single try that either works or doesn't. The one rule that should never bend: nothing in a ticket's own text should ever be able to approve a refund by itself — that's already true today, and any future change to that rule should go through review, not get quietly edited by the agent.